Búsqueda Semántica de Objetos en Simulación

Documento de implementación y guía de pruebas

Heatmaps precisos · Filtro de coherencia · Mapper LLM · YOLOE · Métodos M1–M3

Mario García Blasco

26 de marzo de 2026

Índice

1. Resumen del sistema **2**

2. Características implementadas **2**
2.1. Heatmaps precisos (pipeline de index_map) 2
2.2. Filtro de coherencia del heatmap 2
2.3. Mapper LLM: traducción de objetos a categorías VLMap 3
2.4. YOLOE: verificador visual en llegada 3
2.5. Análisis de regiones del heatmap (extract_heatmap_regions) 4
2.6. Métodos de búsqueda M1, M2, M3 4

3. Cómo probar cada característica **5**
3.1. Requisitos previos 5
3.2. Probar los heatmaps precisos 5
3.3. Probar el filtro de coherencia 5
3.4. Probar el Mapper LLM 5
3.5. Probar YOLOE 6
3.6. Probar el análisis de regiones 6
3.7. Probar los métodos M1, M2, M3 6

4. Pipeline completo: ejemplo de sesión **7**

5. Estructura de ficheros relevantes **7**

6. Próximos pasos **8**

7. Actualización — 2 de abril de 2026 **8**
7.1. Postproceso de heatmaps: `postprocess_heatmap` 8
7.2. Exploración 360° tras fallo de detección 9
7.3. Ruta alternativa (`navigate_to_alternative`) 9
7.4. Archivos modificados 9

1. Resumen del sistema

El sistema implementa un pipeline de tres niveles para la búsqueda de objetos en entornos simulados Matterport3D con Habitat-Sim:

1. **Nivel 1 – Habitaciones (LabelMe):** polígonos de habitaciones etiquetados manualmente que permiten al robot navegar hacia una habitación concreta.
2. **Nivel 2 – Mobiliario (VLMaps):** mapa semántico con 37 categorías MP3D que produce heatmaps de dónde es probable encontrar un mueble.
3. **Nivel 3 – Verificación visual (YOLOE):** detector open-vocabulary que confirma si el objeto específico está presente al llegar al destino.

El fichero principal es:

```
third_party/vlmaps/application/interactive_object_nav.py
```

2. Características implementadas

2.1. Heatmaps precisos (pipeline de index_map)

Problema original: La versión de heatmap usada en navegación interactiva re-normalizaba el mapa CLIP, produciendo regiones muy ruidosas y difusas con falsos positivos.

Solución: Se reemplazó el cálculo por el mismo pipeline que usa `index_map.py` del repositorio original:

1. `vlmap.init_categories(mp3dcat[1:-1])` — inicializa scores CLIP para las 37 categorías.
2. `vlmap.index_map(category, with_init_cat=True)` — genera máscara booleana 3D.
3. `pool_3d_label_to_2d(mask, grid_pos, gs)` — proyecta a vista cenital 2D.
4. `get_heatmap_from_mask_2d(mask_2d, cell_size=0.05, decay_rate=0.01)` — genera heatmap suavizado.

Función clave: `compute_heatmap(robot, category)` en `interactive_object_nav.py`.
Efecto medido:

Categoría	Celdas > 0,5 (antiguo)	Celdas > 0,5 (nuevo)
sofa	22 000	8 000
table	36 000	11 000
toilet	18 000	5 000

El nuevo heatmap es 2–3× más concentrado en las zonas reales.

2.2. Filtro de coherencia del heatmap

Si el heatmap está muy fragmentado (el objeto apareció en muchas celdas dispersas), no es fiable. Se descarta y se usa fallback a BFS.

Función: `_heatmap_is_coherent(mask_2d, min_total_cells=30, min_dominance=0.30)`
Criterios:

- Al menos 30 celdas activas en total.

- La componente conexa más grande ocupa $\geq 30\,\%$ del total de celdas activas (dominio de la componente principal).

Resultados medidos:

Categoría	Dominancia	¿Aceptado?
sofa	44 %	✓
bed	41 %	✓
toilet	45 %	✓
wall	13 %	×
chair	12 %	×
table	16 %	×

2.3. Mapper LLM: traducción de objetos a categorías VLMap

Problema: VLMaps solo conoce 37 categorías MP3D (muebles genéricos). Si el usuario pide "laptop", VLMaps no tiene esa categoría.

Solución: Un LLM (GPT con few-shot) traduce el objeto a una lista de categorías VLMap donde probablemente se encuentre.

Función: `map_object_to_vlmap_categories(object_name)` en `vlmaps/utils/llm_utils.py`

Ejemplos de mapeo:

Objeto pedido	Categorías VLMap devueltas
laptop	table, counter, shelving
book	shelving, table, bed
mug	counter, table
toilet paper	toilet, shelving
tv	tv_monitor, furniture
sofa	sofa (directo, ya está en MP3D)

Lógica:

1. Si el objeto ya es una categoría MP3D → devuelve directamente esa categoría.
2. Si no → llama al LLM con few-shot prompting y parsea JSON `{categories: [...], reason: "..."}`.
3. Fallback: si el parsing falla → devuelve `["furniture", .ºbjects"]`.

2.4. YOLOE: verificador visual en llegada

Al llegar a un candidato, el robot gira 360° y toma fotos con la cámara frontal. YOLOE verifica si el objeto original está visible.

Modelo: YOLOE-11L-Seg cargado desde `/workspace/yoloe-11l-seg.pt`

Funciones:

- `_load_yoloe()` — carga el modelo una sola vez (singleton).
- `yoloe_verify(robot, object_name, conf_thresh=0.25)` — captura frames frontales, ejecuta YOLOE con `set_classes([object_name])`, devuelve `{found, detections, frame_bgr}`.

API de YOLOE (importante):

```
model.set_classes(["laptop"])    # fijar clase ANTES de predict
results = model.predict(frame, conf=0.25)
```

Nota: la primera llamada a `set_classes` descarga el backbone MobileCLIP (~572 MB) – solo ocurre una vez.

Resultados en escena real (6200 frames):

Objeto	Conf. media	¿Detectado correctamente?
sofa	0.258	✓
bed	0.283	✓
chair	0.31	✓

2.5. Análisis de regiones del heatmap (extract_heatmap_regions)

Segmenta el heatmap en regiones candidatas y las ordena por calidad.

Función: `extract_heatmap_regions(heatmap, score_thresh=0.3, min_area=20, min_mean_score=0.3 min_density=0.4)`

Para cada región calcula:

- **área**: número de celdas activas.
- **mean_score**: media del heatmap en esas celdas.
- **density**: fracción de celdas activas vs. bounding box.
- **quality**: $0{,}3 \cdot \hat{a} + 0{,}4 \cdot \bar{s} + 0{,}3 \cdot d$ donde $\hat{a}$ es área normalizada.

Devuelve: lista de dicts ordenada por `quality` descendente:

```
[{"centroid": (r, c), "area": 120, "mean_score": 0.62,
  "density": 0.78, "quality": 0.71}, ...]
```

Resultados medidos:

Categoría	Nº regiones	Mejor quality
sofa	27	0.809
toilet	6	0.824
tv_monitor	16	0.861

2.6. Métodos de búsqueda M1, M2, M3

Al iniciar la navegación interactiva, el sistema pregunta qué método usar.

Método	Nombre	Descripción
M1	Random	Selecciona un punto navegable aleatorio. Baseline ingenuo.
M2	Nearest	BFS: candidato más cercano al robot según el heatmap. Si el heatmap no es coherente, usa fallback aleatorio.
M3	Semantic	Usa `extract_heatmap_regions` y va directamente a la región de mayor calidad.

Funciones:

- `select_goal_random(robot)` — M1.
- `select_goal_nearest(robot, furniture_cat, heatmap)` — M2.

- `select_goal_semantic(robot, furniture_cat, heatmap)` — M3.
- `_METHOD_SELECTORS = {"M1": ..., "M2": ..., "M3": ...}` — dispatch.
- `_active_method = "M2"` — valor por defecto.

3. Cómo probar cada característica

3.1. Requisitos previos

1. Iniciar el contenedor Docker con GPU:

```
./docker/run_docker.sh
```

2. Desde el menú principal (`./docker/menu.sh`):
   - Opción **b)** para construir el mapa VLMap (si no existe).
   - Opción **n)** para iniciar navegación interactiva.

3.2. Probar los heatmaps precisos

Al pedir un objeto en navegación interactiva, se imprime en consola:

```
[Heatmap] sofa: 8234 cells > 0
[Heatmap] coherence: dominant_ratio=0.44 -> ACCEPTED
```

Para comparar con el método antiguo, comentar temporalmente el bloque de `vlmap.init_categories` en `compute_heatmap()` y ver que el número de celdas activas se multiplica por 2-3.

3.3. Probar el filtro de coherencia

Pedir al robot que busque "wall" o "floor":

```
You: find the wall
[Heatmap] wall: 45123 cells > 0
[Heatmap] coherence: dominant_ratio=0.13 -> REJECTED (fallback)
[Nav] Using random fallback goal
```

Contraste: pedir "sofa" o "toilet" – el heatmap es aceptado y el robot va directamente.

3.4. Probar el Mapper LLM

En la sesión interactiva, pedir un objeto no-MP3D:

```
You: find the laptop
[Mapper LLM] laptop -> ['table', 'counter', 'shelving'] (reason:
   laptops are typically on tables or counters)
[Nav] Searching for furniture categories: table, counter, shelving
```

Pedir un objeto MP3D directo para ver el path rápido:

```
You: find the sofa
[Mapper LLM] sofa is already an MP3D category -> ['sofa']
```

3.5. Probar YOLOE

Al llegar al primer candidato, YOLOE ejecuta la verificación:

```
[YOLOE] Verifying 'laptop' at arrival...
[YOLOE] FOUND: laptop (conf=0.41) -> SUCCESS
```

O si no lo encuentra:

```
[YOLOE] laptop NOT found at this location, continuing search...
```

La ventana de OpenCV muestra el frame con bounding boxes cuando hay detecciones.
Test unitario rápido (sin Habitat, solo verificar que YOLOE carga):

```
from ultralytics import YOLOE
model = YOLOE("/workspace/yoloe-11l-seg.pt")
model.set_classes(["sofa"])
import numpy as np
img = np.zeros((480, 640, 3), dtype=np.uint8)
results = model.predict(img, conf=0.25, verbose=False)
print("YOLOE␣OK,␣detections:", len(results[0].boxes))
```

3.6. Probar el análisis de regiones

En el REPL de Python dentro del contenedor:

```
import sys
sys.path.insert(0, "/workspace/vlmaps")
from application.interactive_object_nav import (
    compute_heatmap, extract_heatmap_regions
)
# Asumiendo que robot ya est  inicializado en una sesi n de
   navegaci n,
# o cargando un heatmap guardado:
regions = extract_heatmap_regions(heatmap, score_thresh=0.3, min_area
   =20)
for r in regions[:3]:
    print(f"␣␣centroid={r['centroid']}␣␣quality={r['quality']:.3f}")
```

3.7. Probar los métodos M1, M2, M3

Al iniciar la sesión interactiva (`menu.sh` opción n):

```
Search methods: M1 (random), M2 (nearest), M3 (semantic quality)
Select method [M1/M2/M3, default M2]: M3
```

Prueba comparativa sugerida:

1. Escena 2, objeto: "sofa".
2. Ejecutar 3 veces con M1, anotar pasos hasta encontrarlo.
3. Ejecutar 3 veces con M2, comparar.
4. Ejecutar 3 veces con M3, comparar.
5. M3 debería requerir menos pasos que M1, con M2 como intermedio.

4. Pipeline completo: ejemplo de sesión

```
$ ./docker/menu.sh
> n) Interactive navigation

Search methods: M1 (random), M2 (nearest), M3 (semantic quality)
Select method [M1/M2/M3, default M2]: M3

Scene 2 loaded. Robot at (362, 480).

[M3] You: find the laptop

[Mapper LLM] laptop -> ['table', 'counter', 'shelving']

[Heatmap] table: 11230 cells > 0
[Heatmap] coherence: dominant_ratio=0.38 -> ACCEPTED
[Regions] table: 12 regions found, best quality=0.71

[Nav] Going to table region at (401, 512) [quality=0.71]
... (robot navega) ...

[YOLOE] Verifying 'laptop' at arrival...
[YOLOE] laptop NOT found, trying next region...

[Nav] Going to table region at (388, 495) [quality=0.68]
... (robot navega) ...

[YOLOE] FOUND: laptop (conf=0.37) -> SUCCESS
Goal reached in 2 attempts, 184 steps.
```

5. Estructura de ficheros relevantes

Fichero	Cambios
`third_party/vlmaps/application/interactive_object_nav.py`	Todas las funciones nuevas: `compute_heatmap`, `_heatmap_is_coherent`, `find_heatmap_goal`, `extract_heatmap_regions`, `_load_yoloe`, `yoloe_verify`, `select_goal_random/nearest/semantic`, `_METHOD_SELECTORS`
`third_party/vlmaps/vlmaps/utils/llm_utils.py`	`map_object_to_vlmap_categories()` con few-shot LLM
`third_party/vlmaps/vlmaps/robot/habitat_lang_robot.py`	`_NAV_MARGIN_PX = 2` (margen de 2px en mapa erosionado ≈10 cm)

6. Próximos pasos

1. **M4**: scoring híbrido (VLMap × YOLOE confidence) con re-routing si YOLOE no detecta tras k visitas.
2. **M5**: planificación de 4 viewpoints por región + exploración de fronteras para regiones no alcanzadas.
3. **Benchmark**: `run_evaluation.py` con 10 objetos × 3 escenas × 5 métodos (M1–M5), métricas: success rate, avg steps, avg path length.
4. **ROS 1 Noetic**: wrapper para Toyota HSR (última prioridad).

7. Actualización — 2 de abril de 2026 (segunda iteración)

7.1. YOLOE persistente: `YoloeSession`

Problema

Cada llamada a `yoloe_verify()` lanzaba un subproceso nuevo que cargaba el modelo YOLOE desde cero (~30 s). Durante el scan 360° se producía un `TimeoutExpired` a los 30 s por llamada.

Solución: worker persistente + hilo de fondo

Nuevo fichero `_yoloe_persistent_worker.py`:

```
# Protocolo stdin/stdout (line-buffered):
# arranque  -> imprime "READY"
# petición  <- lee "input_path|output_path"
# respuesta -> imprime "1" (found) o "0"
model = YOLO(weights); model.set_classes([target])
print("READY", flush=True)
for line in sys.stdin:
    input_path, output_path = line.strip().split("|")
    results = model.predict(cv2.imread(input_path), ...)
    cv2.imwrite(output_path, results[0].plot())
    print("1" if found else "0", flush=True)
```

Clase `YoloeSession` en `yoloe_utils.py`:

- `start()`: lanza el subproceso persistente, espera `READY`.
- `check(frame)`: envía frame vía `stdin`, espera resultado. Tras la carga inicial (∼10 s), cada inferencia cuesta ~100–500 ms.
- `start_bg_thread()` / `push_frame()` / `poll_result()`: API asíncrona para el scan en tiempo real. El hilo de fondo consume frames de una `Queue` y publica resultados sin bloquear el bucle principal.
- `get_session(target)`: caché global, reutiliza la sesión si el target no cambia.

7.2. Scan 360° en tiempo real

Antes

`robot.turn(20)` + `yoloe_verify(frame)` bloqueante ⇒ 18 llamadas × 30 s = timeout.

Ahora

1. Pasos nativos de 5° (`robot.sim.step("turn_right")`): la imagen se actualiza en cada paso (72 frames por rotación completa).
2. Cada frame se envía a `YoloeSession.push_frame()` (no bloqueante).
3. `poll_result()` lee el último resultado disponible sin esperar.
4. El display corre a la velocidad del simulador (~30–60 fps); las anotaciones YOLOE aparecen en cuanto el hilo de fondo las produce (~2–5 fps efectivos).
5. Al detectar el objeto, la rotación se detiene inmediatamente.

7.3. Cámara a −30 e inicio más cercano al objeto

- **Tilt cámara**: `habitat_utils.py` actualizado de $-\pi/8$ (22,5°) a $-\pi/6$ (30°) en ambos sensores de color.
- **Standoff reducido**: de 0,5 m a 0,25 m como distancia objetivo. El navmesh snappa el destino al punto navegable más cercano si cae dentro de un obstáculo.
- **Stage de retroceso a 1 m eliminado**: no tiene sentido alejarse si el objeto no se detecta; el scan 360° es más efectivo.

7.4. Flujo de verificación actualizado

Stage	Posición	YOLOE	Si falla →
1	0,25 m standoff	Síncrono	Stage 2
2	360° scan in situ	Asíncrono (hilo bg)	Stage 3
3	Alt. componente heatmap	Síncrono	Desistir

8. Actualización — 2 de abril de 2026 (primera iteración)

8.1. Postproceso de heatmaps: `postprocess_heatmap`

Motivación

Al generar mapas de similitud semántica para queries como *bed*, *window* o *bathroom*, aparece una región principal coherente junto con múltiples activaciones pequeñas dispersas (ruido). El postproceso las elimina de forma genérica, sin reglas por clase.

Pipeline implementado

1. **Suavizado Gaussiano** (`blur_ksize=5`, `blur_sigma=1.0`): reduce ruido de alta frecuencia.
2. **Umbral relativo**: `thr = rel_thresh * max(smoothed)`, con `rel_thresh=0.5`. Adaptativo a cada query y escena.
3. **Componentes conectadas** (8-conectividad, OpenCV `connectedComponentsWithStats`).
4. **Puntuación por componente**: score $= \bar{v} \cdot \ln(1 + A)$ (`mean_log_area`), donde $\bar{v}$ es el valor medio y A el área. Alternativa: `max_sqrt_area`.
5. **Filtro relativo**: se retienen componentes con score $\geq$ `keep_ratio`$\cdot$best_score, con `keep_ratio=0.2`. Preserva objetos pequeños reales con alta similitud.

API:

```
cleaned_heatmap, final_mask, kept_components = postprocess_heatmap(
    heatmap, blur_ksize=5, blur_sigma=1.0, rel_thresh=0.5,
    min_area=3, score_mode="mean_log_area", keep_ratio=0.2)
```

`compute_heatmap` ahora devuelve `(heatmap, kept_components)`.

Coste computacional

Gaussian blur + connected components sobre un mapa 1000×1000 en NumPy/OpenCV: ≈5–15 ms en CPU. Apto para uso interactivo.

8.2. Exploración 360° tras fallo de detección

Flujo

Después de Stage 1 (0,5 m) y Stage 2 (1 m) sin confirmación YOLOE, se ejecuta `scan_360_and_verify`:

1. El robot rota 360 en 18 pasos de 20° cada uno.
2. Cada paso usa `robot.turn(20)`, que internamente ejecuta cuatro acciones discretas `turn_right` de 5°. **No hay teletransporte ni saltos bruscos.**
3. Tras cada giro se captura el frame de cámara y se ejecuta YOLOE.
4. Si YOLOE confirma el objeto, la exploración termina.

8.3. Ruta alternativa (`navigate_to_alternative`)

Si la exploración 360° también falla, el sistema selecciona la componente del heatmap post-procesado con mayor score cuyo centroide esté a ≥ 20 celdas (≈1 m) de la posición actual, planifica un standoff de 1 m sobre la región del VLMap más cercana y navega hasta allí mediante `robot.move_to()` (acciones discretas). Tras llegar, se ejecuta una última verificación YOLOE.

Resumen del flujo completo (Stage 1–4)

Stage	Posición	YOLOE	Si falla →
1	0,5 m standoff	Sí	Stage 2
2	1 m standoff	Sí	Stage 3
3	360° scan (misma pos.)	Sí (×18)	Stage 4
4	Alt. componente heatmap	Sí	Desistir

8.4. Archivos modificados

Archivo	Cambio
`application/interactive_object_nav.py`	`postprocess_heatmap`, `scan_360_and_verify`, `navigate_to_a`